Python Engineering - NumPy Concept Guide

ndim, shape y size: dimension fisica vs dimension de un array

La confusion principal viene de que usamos la palabra **dimension** para dos ideas distintas: la dimension fisica/matematica del problema y la dimension estructural de un array NumPy.

Idea mental clave: en NumPy, `ndim` indica cuantos indices necesitamos para localizar un elemento del array.

1. Un vector fisico 3D puede tener ndim = 1

```
v = np.array([10.0, 20.0, 30.0])

v.ndim   # 1
v.shape  # (3,)
v.size   # 3
```

Fisicamente podemos interpretar esos tres valores como las componentes x, y, z. Sin embargo, NumPy ve una unica secuencia. Para acceder a un elemento basta un indice: v[0], v[1] o v[2]. Por eso ndim = 1.

2. Una matriz 3x3 tiene ndim = 2

```
R = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1]
])

R.ndim   # 2
R.shape  # (3, 3)
R.size   # 9
```

Aunque R opere sobre un espacio fisico 3D, el array es una tabla de filas y columnas. Para localizar R[1,2] hacen falta dos indices: fila y columna. Por eso ndim = 2.

Importante: shape = (3,3), mientras que size = 9. shape describe el tamano de cada axis; size cuenta todos los elementos.

3. Los ejes fisicos x, y, z NO son los axes de NumPy

```
v = np.array([vx, vy, vz])

# NumPy axis 0 contains the three components
# shape = (3,), ndim = 1
```

x, y y z son componentes con significado fisico. NumPy axis 0 es una direccion estructural del array. No debemos identificar automaticamente 'tres ejes fisicos' con 'ndim = 3'.

4. Una matriz de rotacion 3D sigue teniendo solo dos axes NumPy

```
# axis 0 -> rows
# axis 1 -> columns

R.shape == (3, 3)
R.ndim  == 2
```

El numero 3 de shape=(3,3) nos dice que hay tres filas y tres columnas. El numero 2 de ndim nos dice que la estructura necesita dos indices.

5. Por que np.ndim(R) fallo conceptualmente en EX20

En una matriz de rotacion 2D, R.shape=(2,2) y R.ndim=2. Por casualidad, usar np.eye(R.ndim) produce np.eye(2), que es correcto. Pero para una matriz 3D, R.shape=(3,3) y R.ndim sigue siendo 2.

```
# Wrong generic idea:
I = np.eye(R.ndim)

# Correct for a square rotation matrix:
```

```
I = np.eye(R.shape[0])
```

R.shape[0] devuelve el numero de filas. En una matriz de rotacion cuadrada, ese valor coincide con la dimension del espacio sobre el que actua.

6. Dataset de posiciones 3D

```
positions = np.array([
    [1,  2,  3],
    [4,  5,  6],
    [7,  8,  9],
    [10,11,12]
])

positions.ndim   # 2
positions.shape  # (4, 3)
positions.size   # 12
```

Podemos interpretarlo como 4 puntos y 3 componentes x,y,z por punto. axis 0 selecciona el punto; axis 1 selecciona la componente.

```
positions[2, 1]
# point 2, component y
```

7. Ejemplo realmente tridimensional para NumPy

```
trajectory = np.zeros((100, 4, 3))

trajectory.ndim   # 3
trajectory.shape  # (100, 4, 3)
trajectory.size   # 1200

trajectory[time, object, coordinate]
```

Aqui si necesitamos tres indices: instante, objeto y componente. Por eso NumPy dice ndim = 3.

8. Chuleta mental

Propiedad	Significado
ndim	Numero de axes NumPy / numero de indices necesarios
shape	Tamano de cada axis
size	Numero total de elementos

Objeto	ndim	shape	size
scalar 5	0	()	1
vector fisico 3D [x,y,z]	1	(3,)	3
matriz de rotacion 3x3	2	(3,3)	9
100 posiciones 3D	2	(100,3)	300
100 tiempos x 4 objetos x 3 coords	3	(100,4,3)	1200

Idea final

physical dimension != NumPy ndim

Un vector puede vivir en un espacio fisico 3D y tener ndim=1. Una matriz que transforma ese espacio puede tener shape=(3,3) pero ndim=2. NumPy describe la estructura de almacenamiento; la fisica describe el significado de los datos.